

Travel Go: A Cloud-Based Instant Travel Reservation Platform

Project Overview:

Travel Go aims to streamline the travel planning process by offering a centralized platform to book buses, trains, flights, and hotels. Catering to the growing need for real-time and convenient travel solutions, it utilizes cloud infrastructure. The application employs Flask for backend services, AWS EC2 for scalable hosting, DynamoDB for fast data handling, and AWS SNS for immediate notifications. Users can sign up, log in, search, and book transport and accommodations. Additionally, they can review booking history and pick seats interactively, ensuring a user-friendly and responsive system.

Use Case Scenarios:

Scenario 1: Instant Travel Booking Experience

After logging into Travel Go, the user picks a travel type (bus/train/flight/hotel) and fills in preferences. Flask manages backend operations by retrieving matching options from DynamoDB. Once a booking is confirmed, AWS EC2 ensures swift performance even under high usage. This real-time setup allows users to secure bookings efficiently, even during rush periods.

Scenario 2: Instant Email Alerts

Upon booking confirmation, Travel Go uses AWS SNS to instantly send emails with booking info. For example, when a flight is booked, Flask handles the transaction and SNS notifies the user via email. The booking details are saved securely in DynamoDB. This smooth integration boosts reliability and enhances user confidence.

Scenario 3: Easy Booking Access and Control

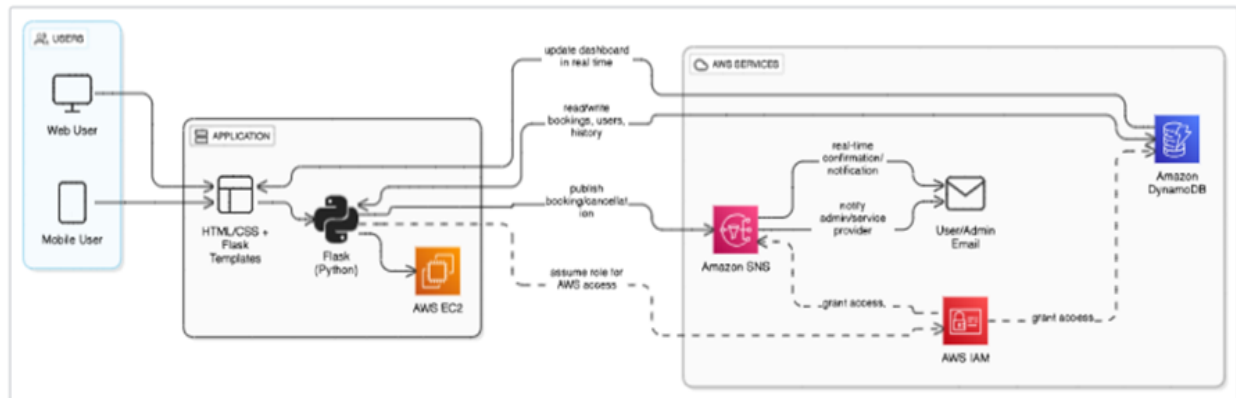
Users can return anytime to manage previous bookings. For instance, a user checks hotel reservations made last week. Flask quickly fetches this from DynamoDB. Thanks to its cloud-first approach, Travel Go provides uninterrupted access, letting users easily cancel or make new plans. EC2 hosting guarantees consistent performance across multiple users.

AWS Architecture Diagram

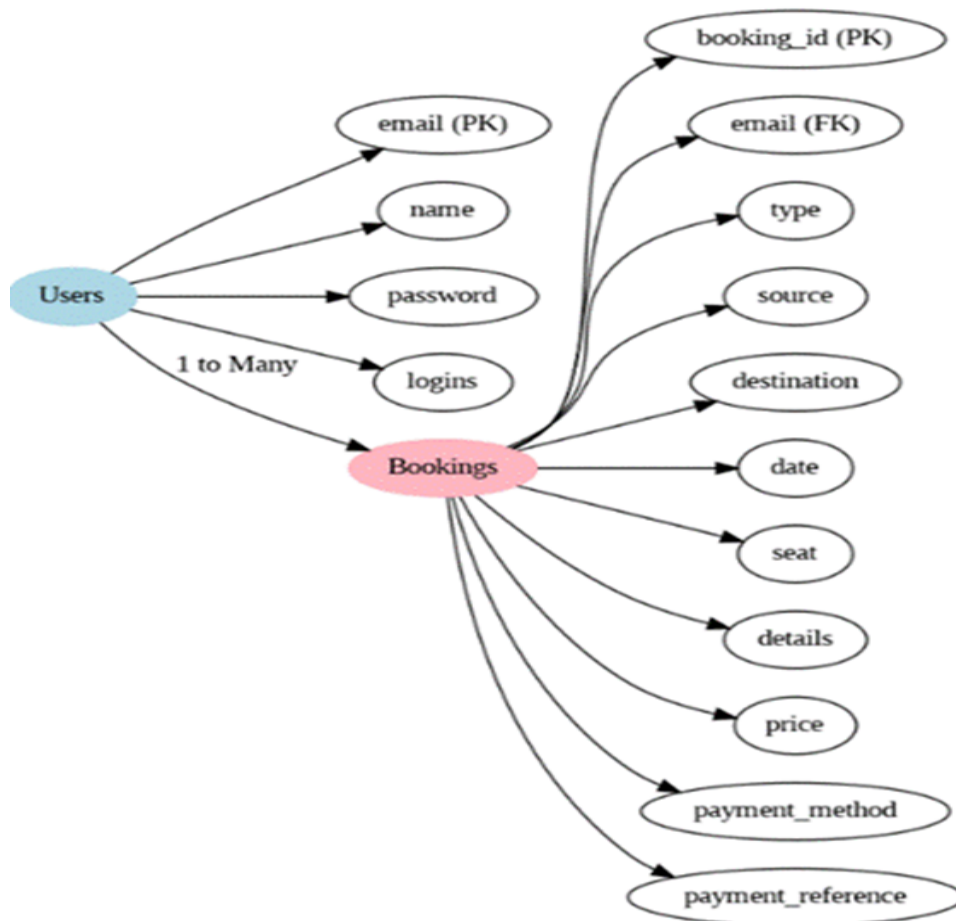
To build a scalable and responsive travel booking platform, a well-structured cloud architecture is essential. Travel Go leverages core AWS services such as EC2, DynamoDB, IAM, and SNS to ensure high availability and fault tolerance. The

architecture is designed to support real-time booking, seamless data flow, and secure operations. Below is the visual representation of the AWS setup used in the project.

AWS Architecture



Entity Relationship Diagram



Requirements:

1. AWS Account Configuration
2. IAM Basics Overview
3. EC2 Introduction
4. DynamoDB Fundamentals
5. SNS Concepts
6. Git Version Control

Development Steps:

1. AWS Account Setup and Login

- Activity 1.1: Register for an AWS account if you haven't already.
- Activity 1.2: Access the AWS Management Console using your credentials.

2. DynamoDB Database Initialization

- Activity 2.1: Create a DynamoDB table.
- Activity 2.2: Define attributes for users and travel bookings.

3. SNS Notification Configuration

- Activity 3.1: Create SNS topics to notify users upon booking.
- Activity 3.2: Add user and provider emails as subscribers for updates.

4. Backend Development using Flask

- Activity 4.1: Build backend logic with Flask.
- Activity 4.2: Connect AWS services using the boto3 SDK.

5. IAM Role Configuration

- Activity 5.1: Set up IAM roles with specific permissions.
- Activity 5.2: Assign relevant policies to each role.

6. EC2 Instance Launch

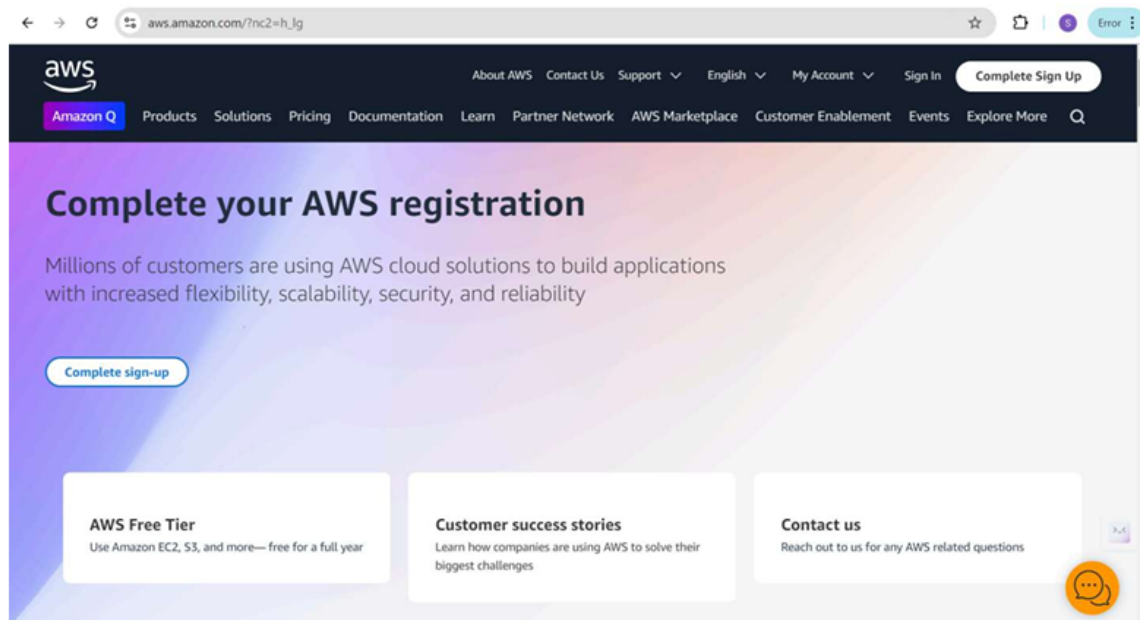
- Activity 6.1: Start an EC2 instance to host the backend.
- Activity 6.2: Set up security rules to allow HTTP and SSH traffic.

7. Flask Application Deployment

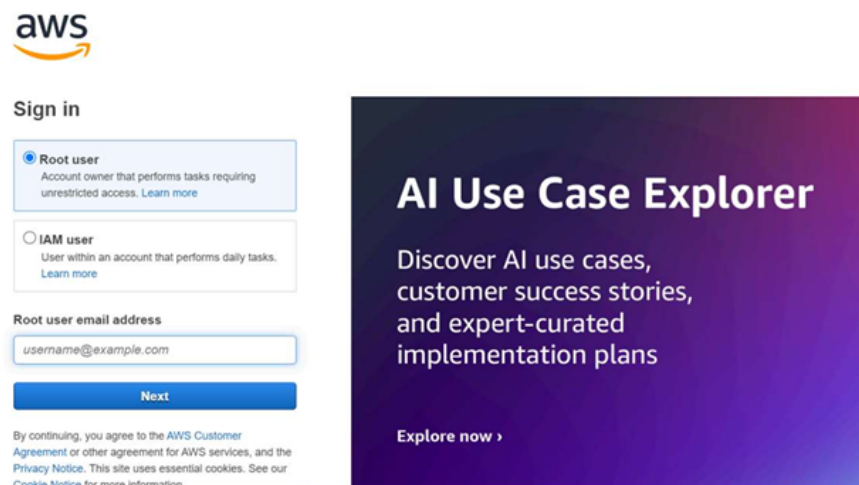
- Activity 7.1: Upload your Flask files to the EC2 instance.
- Activity 7.2: Launch your Flask server from the instance.

8. Testing and Launch

1. AWS Account Setup and Login



2. Log in to the AWS Management Console



After logging into the AWS Console:

Once you're logged in, you can access a wide range of AWS services from the dashboard. Use the search bar at the top to quickly navigate to services like EC2, DynamoDB, SNS, or IAM as needed for your project setup.

3. IAM Roles Creation

The top screenshot shows the AWS IAM Dashboard. The left sidebar contains the 'Identity and Access Management (IAM)' menu with options like 'Dashboard', 'Access management', 'Access reports', and 'Tools'. The main content area displays 'IAM resources' with a red box indicating an 'Access denied' error for the action 'iam:GetAccountSummary'. Another red box on the right shows an 'Access denied' error for the action 'iam:ListAccountAliases'. Both error messages provide the user ARN and context, and include a 'Diagnose with Amazon Q' button.

The bottom screenshot shows the 'Create role' page in the AWS IAM console. The 'Service or use case' dropdown is set to 'EC2'. Under the 'Use case' section, 'EC2' is selected, which allows EC2 instances to call AWS services on your behalf. Other use cases listed include 'EC2 Role for AWS Systems Manager', 'EC2 Spot Fleet Role', 'EC2 - Spot Fleet Auto Scaling', 'EC2 - Spot Fleet Tagging', 'EC2 - Spot Instances', 'EC2 - Spot Fleet', and 'EC2 - Scheduled Instances'. The page has 'Cancel' and 'Next' buttons at the bottom right.

The screenshot shows the AWS IAM console interface during the 'Add permissions' step of creating a role. The breadcrumb navigation at the top indicates the path: IAM > Roles > Create role. On the left sidebar, the 'Add permissions' step is highlighted. The main content area is titled 'Permissions policies (3/1060)' and includes a search bar with the text 'ec2' and a dropdown menu set to 'All types', showing 33 matches. Below this is a table of AWS managed policies. The policy 'AmazonEC2FullAccess' is selected, indicated by a blue checkmark in the first column. Other policies listed include AmazonEC2ContainerRegistryFullAccess, AmazonEC2ContainerRegistryPowerUser, AmazonEC2ContainerRegistryPullOnly, AmazonEC2ContainerRegistryReadOnly, AmazonEC2ContainerServiceAutoscaleRole, AmazonEC2ContainerServiceEventsRole, AmazonEC2ContainerServiceforEC2Role, AmazonEC2ContainerServiceRole, and AmazonEC2ReadOnlyAccess.

	Policy name	Type	Description
☐	AmazonEC2ContainerRegistryFullAccess	AWS managed	Provides administrative access to Ama...
☐	AmazonEC2ContainerRegistryPowerUser	AWS managed	Provides full access to Amazon EC2 Co...
☐	AmazonEC2ContainerRegistryPullOnly	AWS managed	Provides access to pull images from A...
☐	AmazonEC2ContainerRegistryReadOnly	AWS managed	Provides read-only access to Amazon E...
☐	AmazonEC2ContainerServiceAutoscaleRole	AWS managed	Policy to enable Task Autoscaling for A...
☐	AmazonEC2ContainerServiceEventsRole	AWS managed	Policy to enable CloudWatch Events fo...
☐	AmazonEC2ContainerServiceforEC2Role	AWS managed	Default policy for the Amazon EC2 Rol...
☐	AmazonEC2ContainerServiceRole	AWS managed	Default policy for Amazon ECS service ...
☒	AmazonEC2FullAccess	AWS managed	Provides full access to Amazon EC2 via...
☐	AmazonEC2ReadOnlyAccess	AWS managed	Provides read only access to Amazon E...

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

Studentuser

Maximum 64 characters. Use alphanumeric and '+,=, @, _' characters.

Description
Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/\|()!#\$%*~0-~

Step 1: Select trusted entities

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "sts:AssumeRole"
7     }
8   ]
9 }

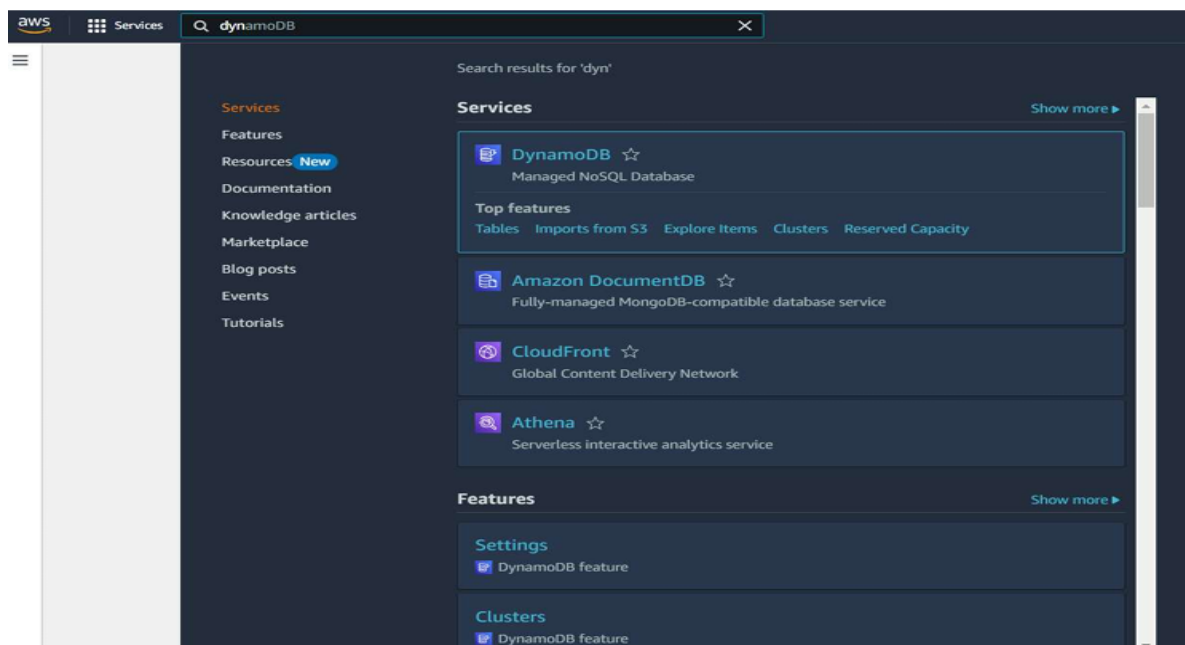
```

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

4. DynamoDB Database Creation and Setup

Familiarize yourself with the DynamoDB service interface. DynamoDB offers a fast and flexible NoSQL database ideal for serverless applications like Travel Go. You will now proceed to create tables to store user and booking data. In this setup, you'll define partition keys to uniquely identify records—such as using “Email” for users or “Train Number” for trains. Properly configuring your attributes is crucial to ensure efficient querying and data retrieval. DynamoDB’s schema-less design allows flexibility while maintaining performance, making it well-suited for handling diverse booking data types in Travel Go.

- In the AWS Console, navigate to DynamoDB and click on create tables



Create a DynamoDB table for storing registration details and book Requests

1. Create Users table with partition key “Email” with type String and click on create tables.

The screenshot shows the AWS Management Console interface for creating a new DynamoDB table. The browser address bar indicates the URL is `us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#create-table`. The page title is "Create table".

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (`_`), hyphens (`-`), and periods (`.`).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

String

1 to 255 characters and case sensitive.

Table settings

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Repeat the same procedure to create additional tables:

- **Train Table:** Use "Train Number" as the partition key to uniquely identify each train.
- **Bookings Table:** Set "User Email" as the partition key and "Booking Date" as the sort key to efficiently organize and retrieve individual user booking records based on date.

The screenshot shows the AWS Management Console interface for creating a new DynamoDB table. The browser address bar indicates the URL is `us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#create-table`. The page title is "Create table".

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (`_`), hyphens (`-`), and periods (`.`).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

String

1 to 255 characters and case sensitive.

Table settings

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#create-table

DynamoDB > Tables > Create table

Share your feedback on Amazon DynamoDB

Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.

Share feedback

Notifications 0 0 1 1 0 0

Create table

Table details

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

trains

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

train_number

String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

date

String

1 to 255 characters and case sensitive.

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

DynamoDB > Tables

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Reserved capacity

Settings

DAX

Clusters

Subnet groups

Parameter groups

Events

Tables (3)

Find tables

Any tag key

Any tag value

< 1 >

bookings

Active

user_email (S)

booking_date (S)

0

0

Off

On-d

trains

Active

train_number (S)

date (S)

0

0

Off

On-d

travelgo_users

Active

email (S)

-

0

0

Off

On-d

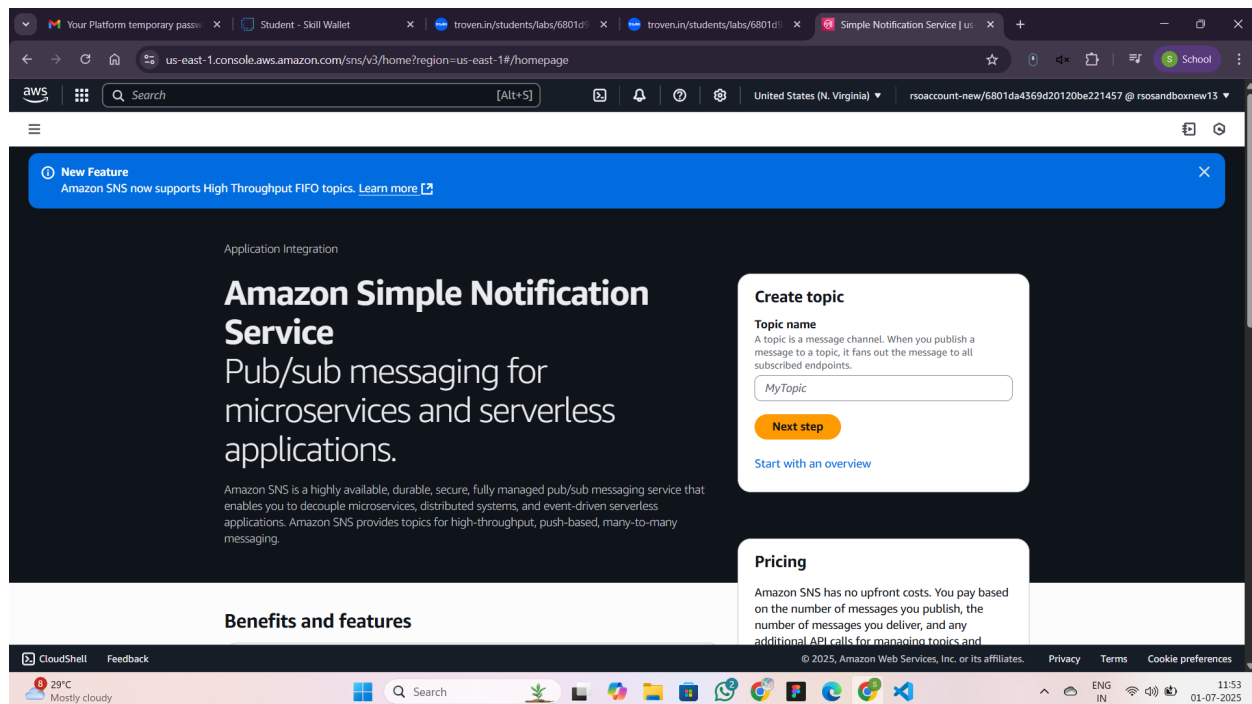
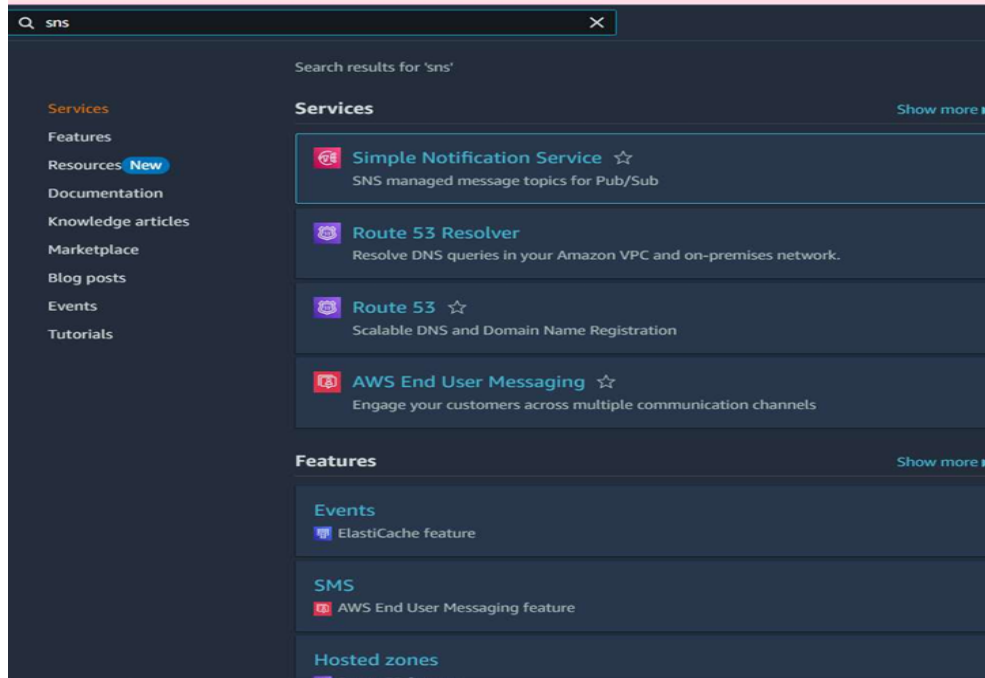
CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

5. SNS Notification Setup

1. Create SNS topics for sending email notifications to users.



1. Click on Create Topic and choose a name for the topic.

The screenshot shows the AWS SNS 'Create topic' console. The browser address bar indicates the URL is `us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/create-topic`. The page title is 'Create topic'. The 'Details' section is active, showing the 'Type' as 'Standard' (selected) and 'Name' as 'Travelgq'. The 'Display name' is 'My Topic'. The 'Encryption' section is optional and shows that Amazon SNS provides in-transit encryption by default.

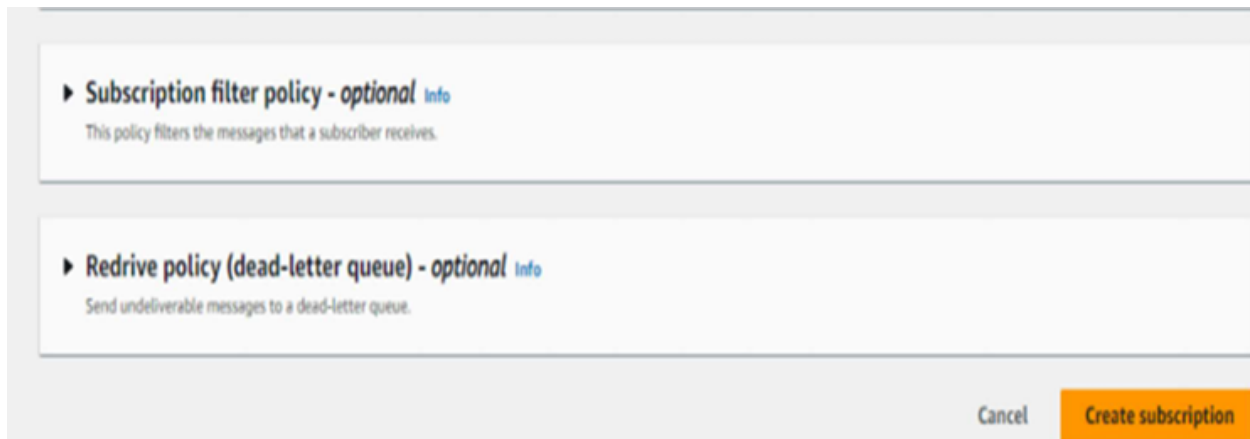
2. Click on create topic

To begin configuring notifications, navigate to the SNS dashboard and click on “Create Topic.” Choose the topic type (Standard or FIFO) based on your requirements. Provide a meaningful name for the topic that reflects its purpose (e.g., booking-alerts). This topic will serve as the communication channel for sending notifications to subscribed users.

The screenshot shows the 'Create topic' console page with the 'Details' section expanded. The 'Access policy' section is optional and defines who can access the topic. The 'Data protection policy' section is optional and defines which sensitive data to monitor. The 'Delivery policy (HTTP/S)' section is optional and defines how Amazon SNS retries failed deliveries. The 'Delivery status logging' section is optional and configures the logging of message delivery status. The 'Tags' section is optional and allows assigning metadata labels to the topic. The 'Active tracing' section is optional and allows using AWS X-Ray active tracing for the topic.

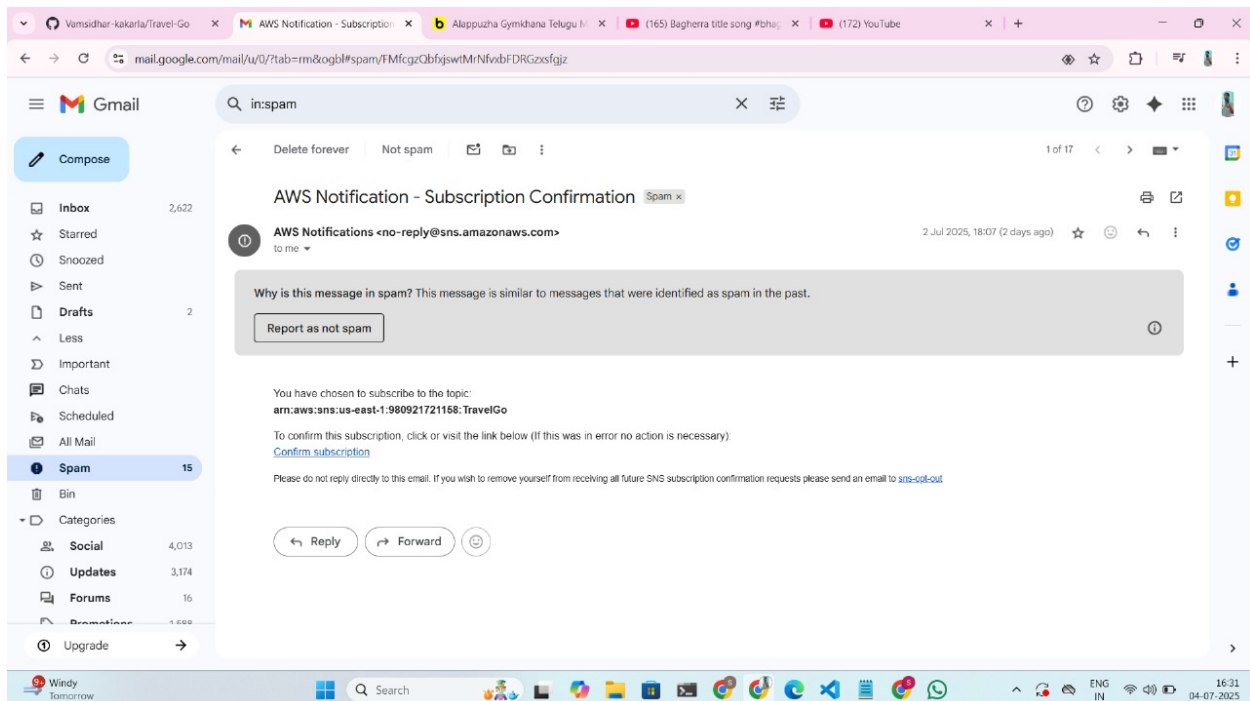
3. Configure the SNS topic and note down the **Topic ARN**.

4. Click on create subscription.



5. Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail

Once the email subscription is confirmed, the endpoint becomes active for receiving notifications. This ensures that users will instantly get booking confirmations or alerts. You can manage or remove subscriptions anytime via the SNS dashboard. It's recommended to test the setup by publishing a sample message to verify successful delivery.



Backend Configuration and Coding


```

1  from flask import Flask, render_template, request, redirect, url_for, session, flash, jsonify
2  from pymongo import MongoClient
3  from werkzeug.security import generate_password_hash, check_password_hash
4  from datetime import datetime
5  from bson.objectid import ObjectId
6  from bson.errors import InvalidId

```

- Flask: Initiates web application
- render_template: Loads Jinja2 HTML templates
- request: Collects input from forms/API
- redirect/url_for: Page redirection logic
- session: Keeps user-specific info
- jsonify: Returns data in JSON structure

```

8  app = Flask(__name__)

```

Begin building the web application by initializing the Flask app instance using `Flask(__name__)`, which sets up the core of your backend framework.

```

18  users_table = dynamodb.Table('travelgo_users')
19  trains_table = dynamodb.Table('trains') # Note: This table is declared but not used in the provided routes.
20  bookings_table = dynamodb.Table('bookings')

```

SNS connection:

This function sends alerts using AWS SNS by publishing a subject and message to a predefined topic. It uses `sns_client.publish()` with error handling to catch failures and print debug info. This ensures users receive real-time booking notifications.

```

22  SNS_TOPIC_ARN = 'arn:aws:sns:us-east-1:490004646397:Travelgo:372db6a7-7131-4571-8397-28b62c9e08a8'
23
24  # Function to send SNS notifications
25  # This function is duplicated in the original code, removing the duplicate.
26  def send_sns_notification(subject, message):
27      try:
28          sns_client.publish(
29              TopicArn=SNS_TOPIC_ARN,
30              Subject=subject,
31              Message=message
32          )
33      except Exception as e:
34          print(f"SNS Error: Could not send notification - {e}")
35          # Optionally, flash an error message to the user or log it more robustly.

```

Routes:

☒ Route Overview of TravelGo:

The application begins with user onboarding through the Register and Login routes, securing user access. Once authenticated, users are redirected to the Dashboard, which serves as the central hub. From there, they can explore and book through Bus, Train, Flight, and Hotel modules. Each booking passes through a Confirmation step where details are reviewed before final submission. The Final Confirmation route stores the booking and triggers email alerts. Users also have the option to manage or cancel their reservations via the Cancel Booking route, ensuring full control and flexibility.

Let's take a closer look at the backend code that powers these application routes.

```
38 @app.route('/')
39 def index():
40     return render_template('index.html')
41
42 @app.route('/register', methods=['GET', 'POST'])
43 def register():
44     if request.method == 'POST':
45         email = request.form['email']
46         password = request.form['password']
47
48         # Check if user already exists
49         # This uses get_item on the primary key 'email', so no GSI needed.
50         existing = users_table.get_item(Key={'email': email})
51         if 'Item' in existing:
52             flash('Email already exists!', 'error')
53             return render_template('register.html')
54
55         # Hash password and store user
56         hashed_password = generate_password_hash(password)
57         users_table.put_item(Item={'email': email, 'password': hashed_password})
58         flash('Registration successful! Please log in.', 'success')
59         return redirect(url_for('login'))
60     return render_template('register.html')
```

```

62 @app.route('/login', methods=['GET', 'POST'])
63 def login():
64     if request.method == 'POST':
65         email = request.form['email']
66         password = request.form['password']
67
68         # Retrieve user by email (primary key)
69         user = users_table.get_item(Key={'email': email})
70
71         # Authenticate user
72         if 'Item' in user and check_password_hash(user['Item']['password'], password):
73             session['email'] = email
74             flash('Logged in successfully!', 'success')
75             return redirect(url_for('dashboard'))
76         else:
77             flash('Invalid email or password!', 'error')
78             return render_template('login.html')
79     return render_template('login.html')

```

```

81 @app.route('/logout')
82 def logout():
83     session.pop('email', None)
84     flash('You have been logged out.', 'info')
85     return redirect(url_for('index'))
86
87 @app.route('/dashboard')
88 def dashboard():
89     if 'email' not in session:
90         return redirect(url_for('login'))
91     user_email = session['email']
92
93     # Query bookings for the logged-in user using the primary key 'user_email'
94     # No GSI is needed here as 'user_email' is likely the partition key for the bookings_table.
95     response = bookings_table.query(
96         KeyConditionExpression=Key('user_email').eq(user_email),
97         ScanIndexForward=False # Get most recent bookings first
98     )
99     bookings = response.get('Items', [])
100
101     # Convert Decimal types from DynamoDB to float for display if necessary
102     for booking in bookings:
103         if 'total_price' in booking:
104             try:
105                 booking['total_price'] = float(booking['total_price'])
106             except (TypeError, ValueError):
107                 booking['total_price'] = 0.0 # Default value if conversion fails
108     return render_template('dashboard.html', username=user_email, bookings=bookings)

```

```
110 @app.route('/train')
111 def train():
112     if 'email' not in session:
113         return redirect(url_for('login'))
114     return render_template('train.html')
```

```
116 @app.route('/confirm_train_details')
117 def confirm_train_details():
118     if 'email' not in session:
119         return redirect(url_for('login'))
120
121     booking_details = {
122         'name': request.args.get('name'),
123         'train_number': request.args.get('trainNumber'),
124         'source': request.args.get('source'),
125         'destination': request.args.get('destination'),
126         'departure_time': request.args.get('departureTime'),
127         'arrival_time': request.args.get('arrivalTime'),
128         'price_per_person': Decimal(request.args.get('price')),
129         'travel_date': request.args.get('date'),
130         'num_persons': int(request.args.get('persons')),
131         'item_id': request.args.get('trainId'), # This is the train ID
132         'booking_type': 'train',
133         'user_email': session['email'],
134         'total_price': Decimal(request.args.get('price')) * int(request.args.get('persons'))
135     }
```

```
492 @app.route('/cancel_booking', methods=['POST'])
493 def cancel_booking():
494     if 'email' not in session:
495         return redirect(url_for('login'))
496
497     booking_id = request.form.get('booking_id')
498     user_email = session['email']
499     booking_date = request.form.get('booking_date') # This is crucial as it's the sort key
500
501     if not booking_id or not booking_date:
502         flash("Error: Booking ID or Booking Date is missing for cancellation.", 'error')
503         return redirect(url_for('dashboard'))
504
505     try:
506         # Delete item using the primary key (user_email and booking_date)
507         # This does not use GSI, so it remains unchanged.
508         bookings_table.delete_item(
509             Key={'user_email': user_email, 'booking_date': booking_date}
510         )
511         flash(f"Booking {booking_id} cancelled successfully!", 'success')
512     except Exception as e:
513         flash(f"Failed to cancel booking {booking_id}: {str(e)}", 'error')
514
515     return redirect(url_for('dashboard'))
```

Note: The implementation logic for train booking, train details, and cancellation is consistent across other modules such as bus, flight, and hotel. Each follows a similar structure for handling user input, storing booking data, and managing cancellations using the same backend principles. This modular approach promotes code reusability and simplifies maintenance across the application. Minor adjustments are made in each module to accommodate specific fields like transport type or room preferences. Overall, the core flow—search, select, book, and cancel—remains consistent for all services.

Deployment code:

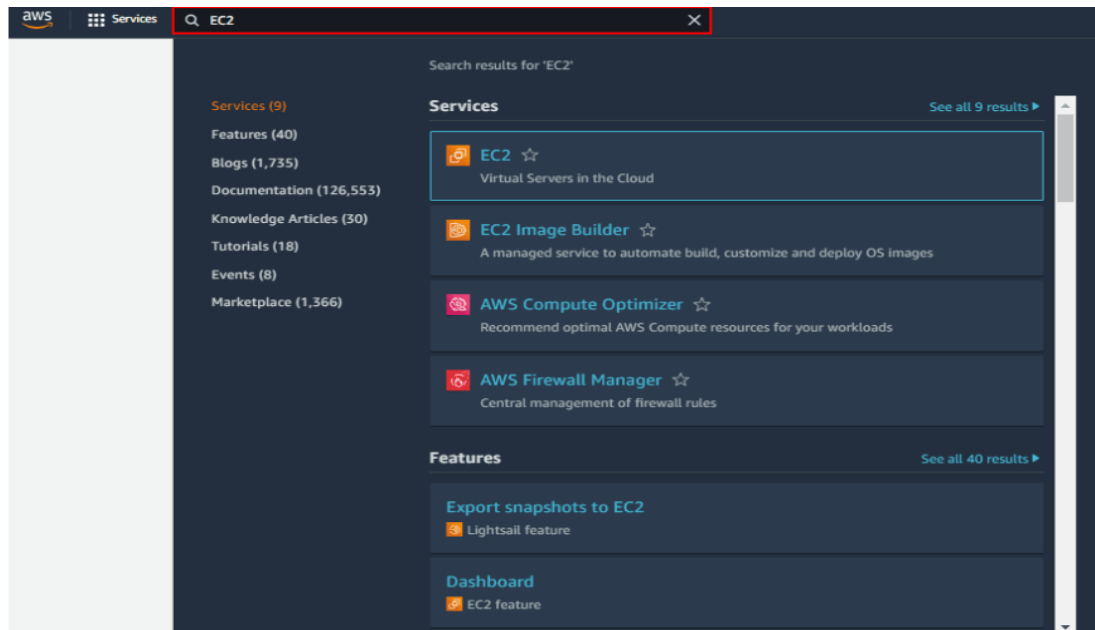
```
518 if __name__ == '__main__':
519     # IMPORTANT: In a production environment, disable debug mode and specify a production-ready host.
520     app.run(debug=True, host='0.0.0.0')
```

☒ Start the Flask server by configuring it to run on all network interfaces (0.0.0.0) at port 5000, with debug mode enabled to support development and live testing.

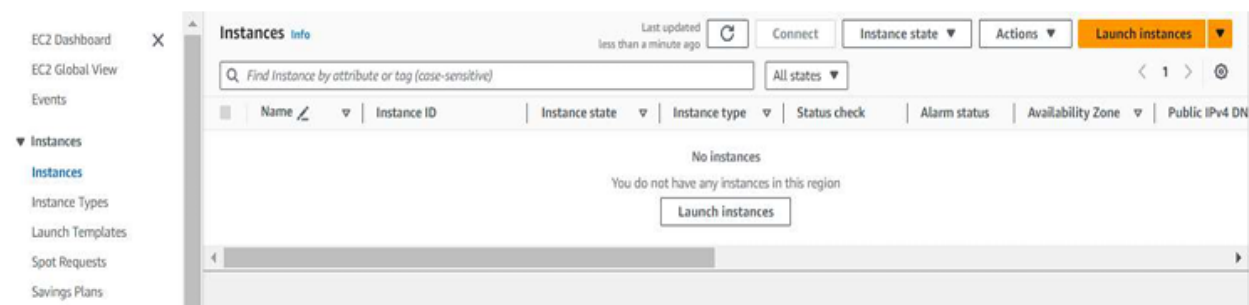
EC2 Instance Setup:

main	1 Branch	0 Tags	Go to file	Add file	Code
Vamsidhar-kakarla Update app.py with latest changes 18bfa3a - 2 days ago 9 Commits					
static	Initial commit		3 days ago		
templates	Initial commit		3 days ago		
venv	Initial commit		3 days ago		
venv_new	Initial commit		3 days ago		
README.md	Initial commit		3 days ago		
app.py	Update app.py with latest changes		2 days ago		

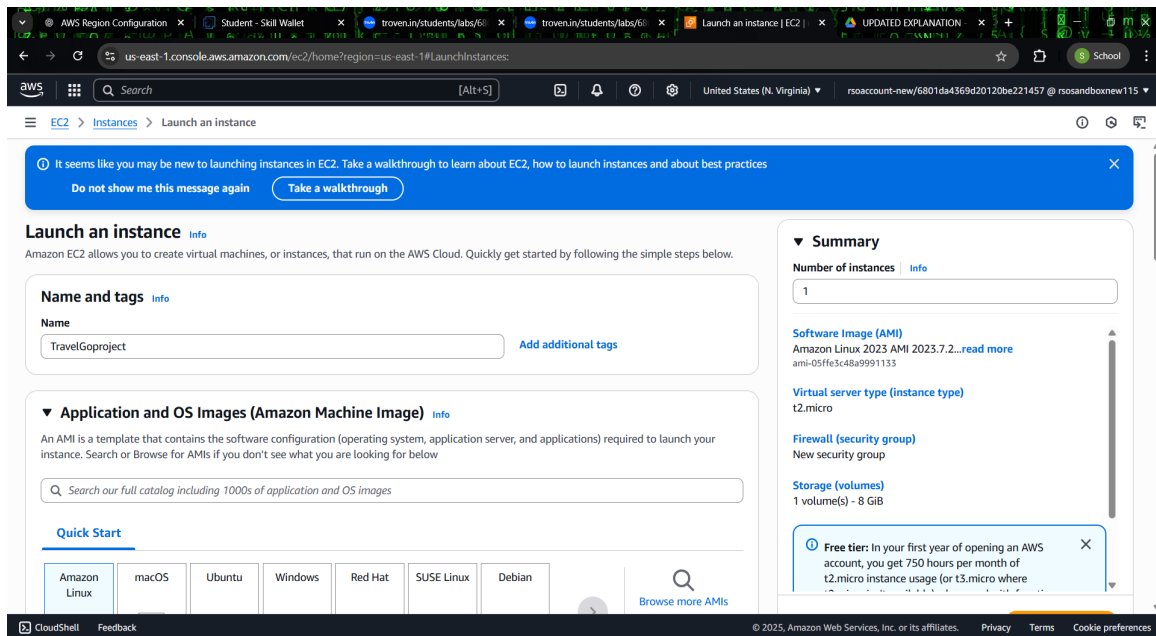
Launch an EC2 instance to host the Flask application.



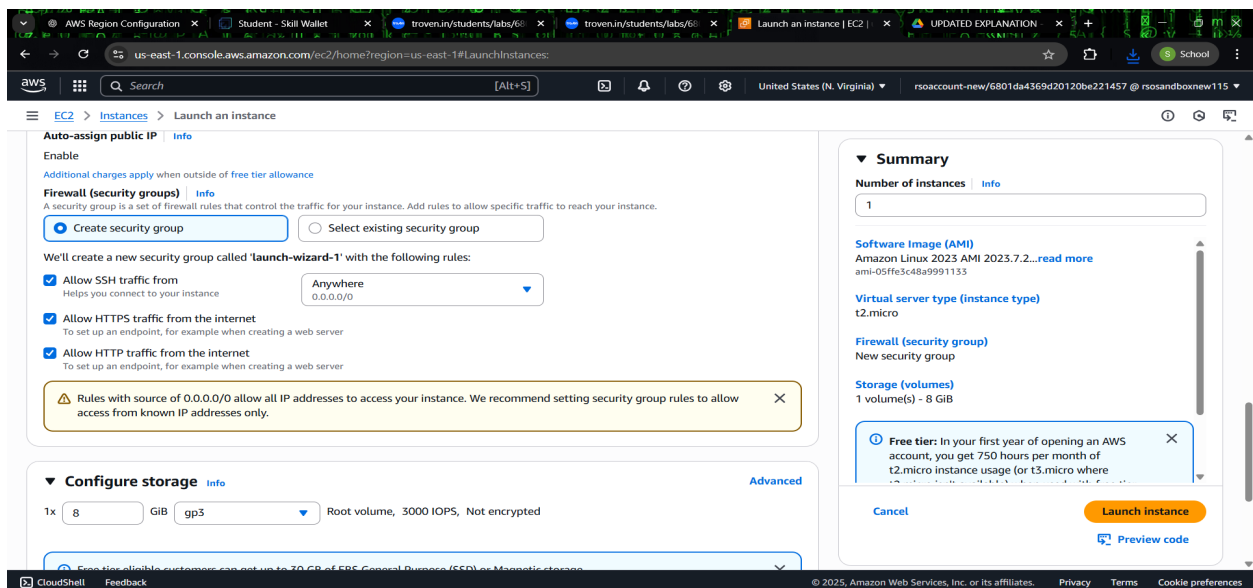
1. Click on launch Instance:

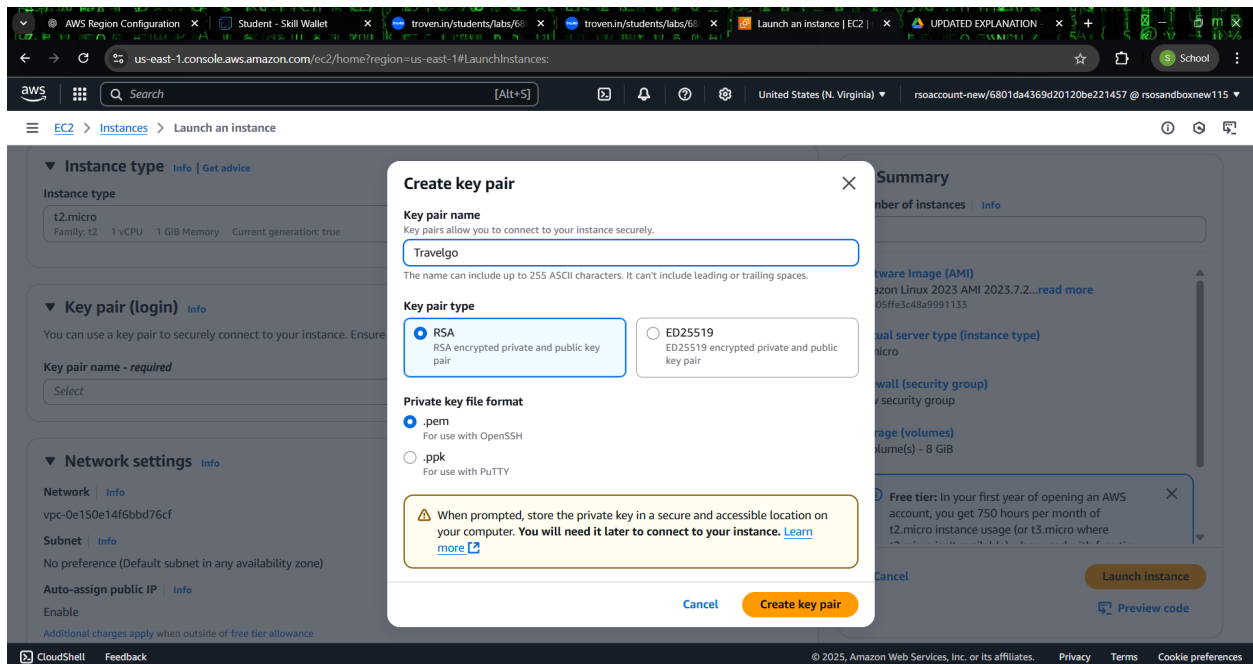


Once the EC2 instance is launched, it acts as the server backbone for your application. Naming the instance as Travelgoproject helps in easy identification during future scaling or monitoring. You can manage performance, logs, and connectivity from the EC2 dashboard. This instance will host and run your Flask backend reliably.

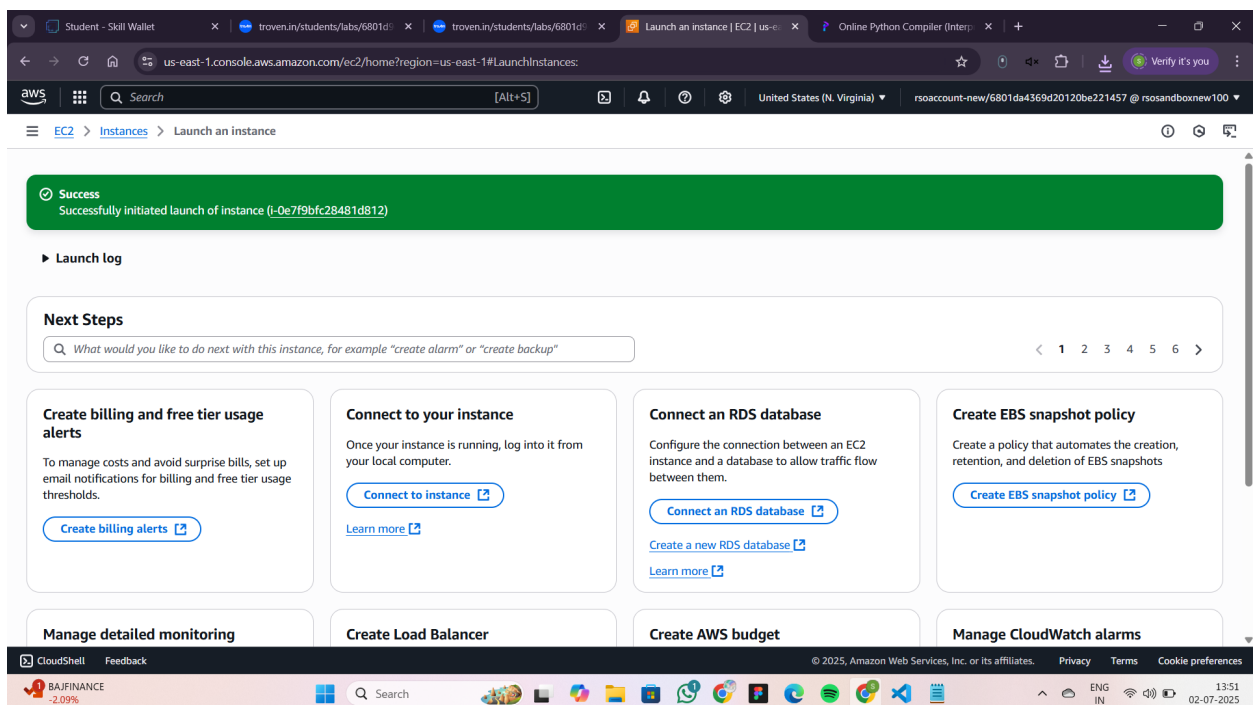


✂ Create a key pair named Travelgo to securely connect to your EC2 instance via SSH. Download and store the .pem file safely, as it will be required for future logins. While setting up the firewall, configure the security group to allow inbound traffic on ports 22 (SSH) and 5000 (Flask). This ensures both secure access and proper functioning of the web application. It's recommended to restrict SSH access to your specific IP range for added security. The Flask port (5000) should be open to all only during development—limit access in production. Properly configured security groups help prevent unauthorized access while keeping your app responsive and accessible.



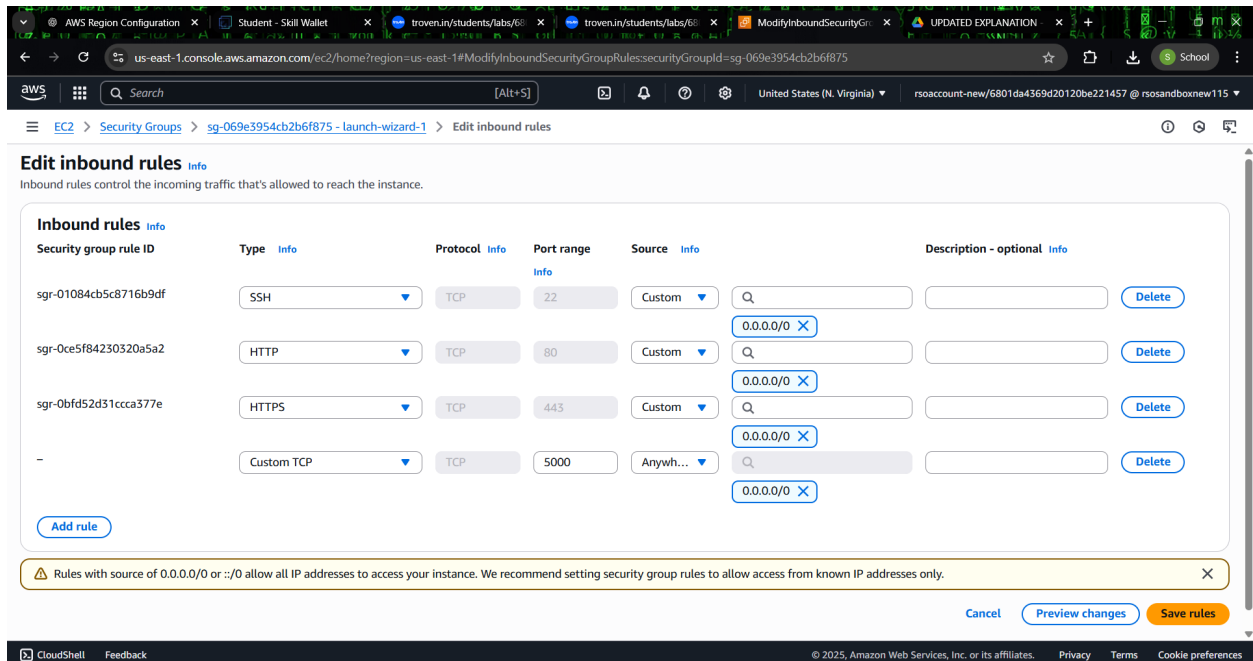


Wait for the confirmation message like “Successfully initiated launch of instance i-0e7f9bfc28481d812,” indicating the instance is being provisioned. During this process, create a key pair named Travelgo (RSA type) with .pem file format. Save this private key securely, as it will be needed for future SSH access.



Edit Inbound Rules:

Select the EC2 instance you just launched and ensure it's in the "running" state. Navigate to the "Security" tab, then click "Edit inbound rules." Add a new rule with the following settings: Type – Custom TCP, Protocol – TCP, Port Range – 5000, Source – Anywhere (IPv4) 0.0.0.0/0. This allows external access to your Flask application running on port 5000.

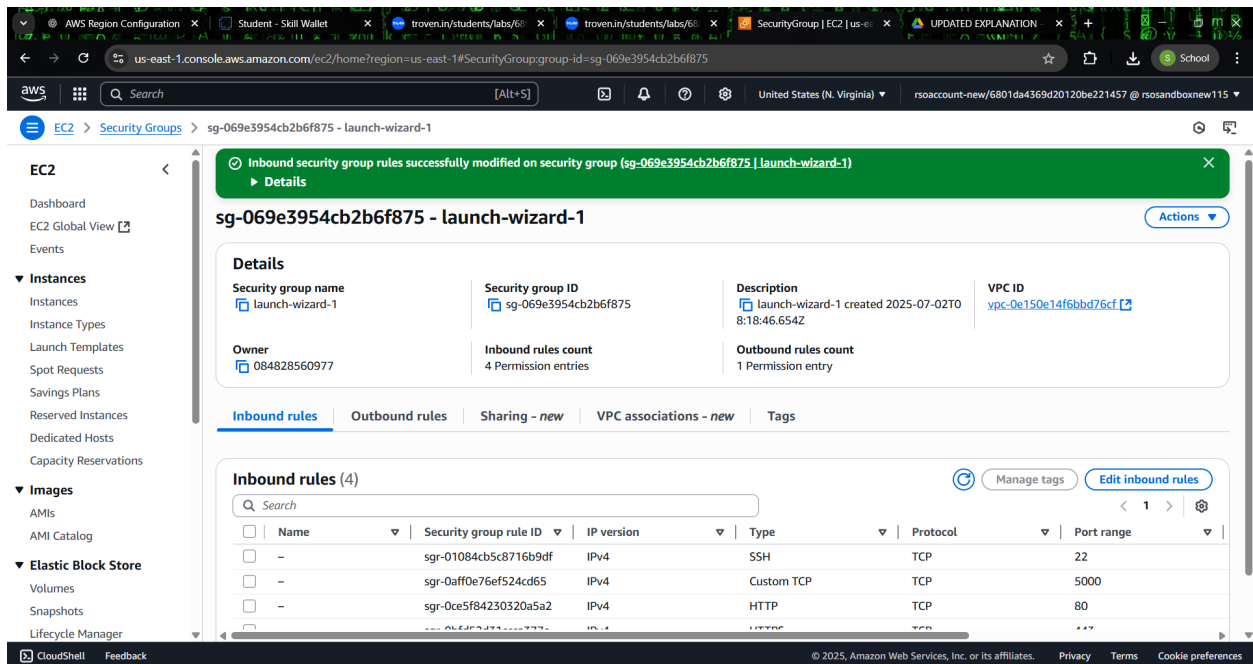


The screenshot shows the AWS Management Console interface for editing inbound rules on a security group. The breadcrumb navigation indicates the path: EC2 > Security Groups > sg-069e3954cb2b6f875 - launch-wizard-1 > Edit inbound rules. The page title is "Edit inbound rules" with an "Info" link. A subtitle states: "Inbound rules control the incoming traffic that's allowed to reach the instance."

The main content area is titled "Inbound rules" and contains a table with the following columns: Security group rule ID, Type, Protocol, Port range, Source, and Description - optional. There are four rules listed:

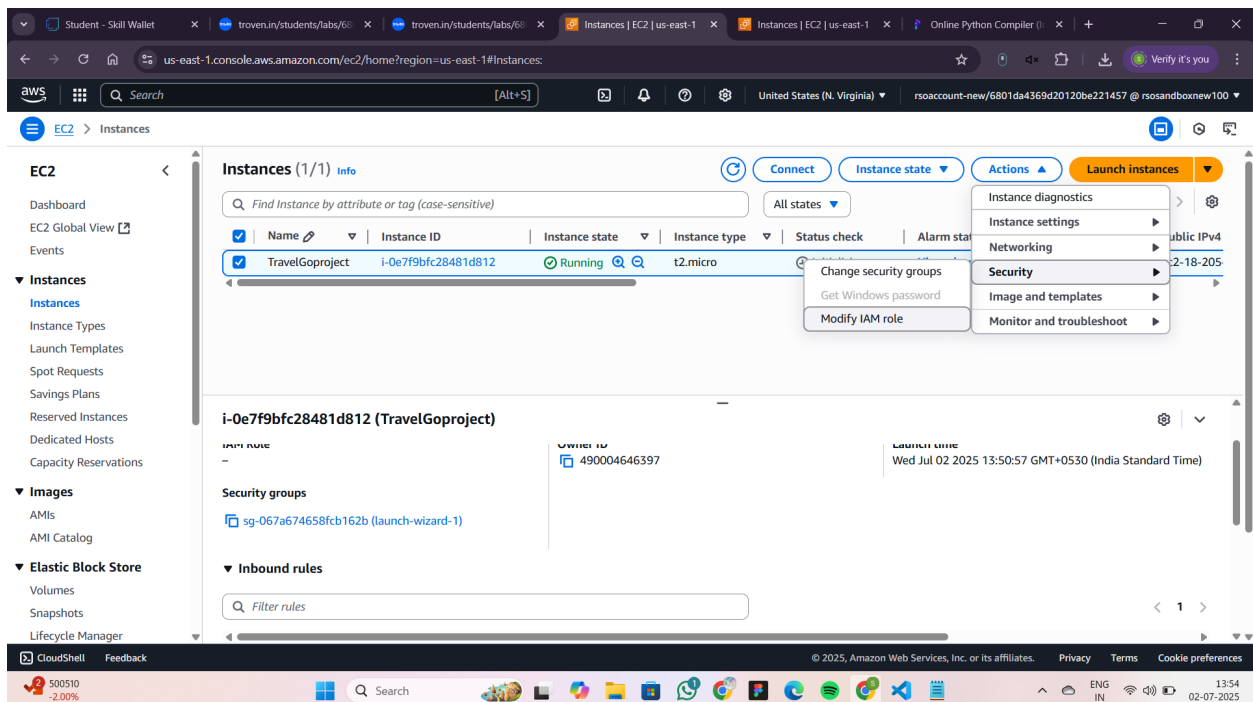
Security group rule ID	Type	Protocol	Port range	Source	Description - optional
sgr-01084cb5c8716b9df	SSH	TCP	22	Custom	
sgr-0ce5f84230320a5a2	HTTP	TCP	80	Custom	
sgr-0bfd52d31ccca377e	HTTPS	TCP	443	Custom	
-	Custom TCP	TCP	5000	Anywhere (0.0.0.0/0)	

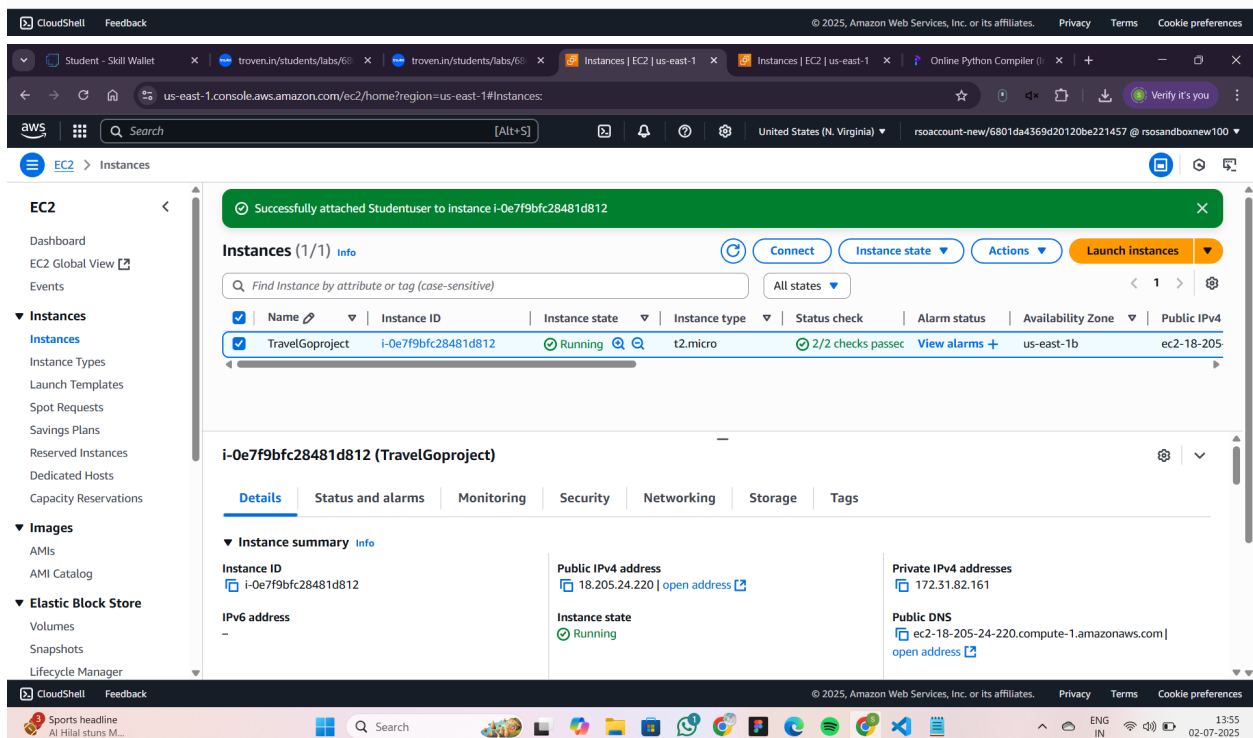
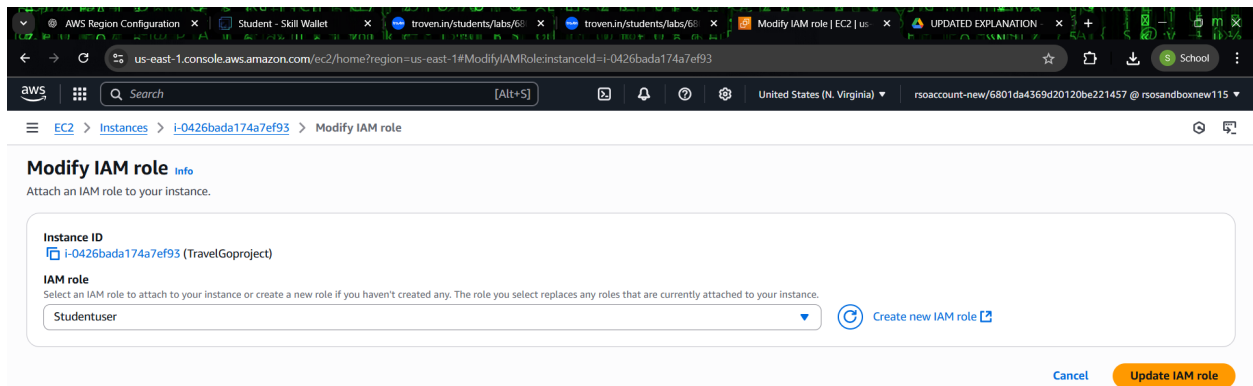
Each rule has a "Delete" button. At the bottom left of the table is an "Add rule" button. Below the table, a yellow warning banner states: "Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only." At the bottom right are "Cancel", "Preview changes", and "Save rules" buttons. The footer includes "CloudShell", "Feedback", and copyright information for Amazon Web Services, Inc.



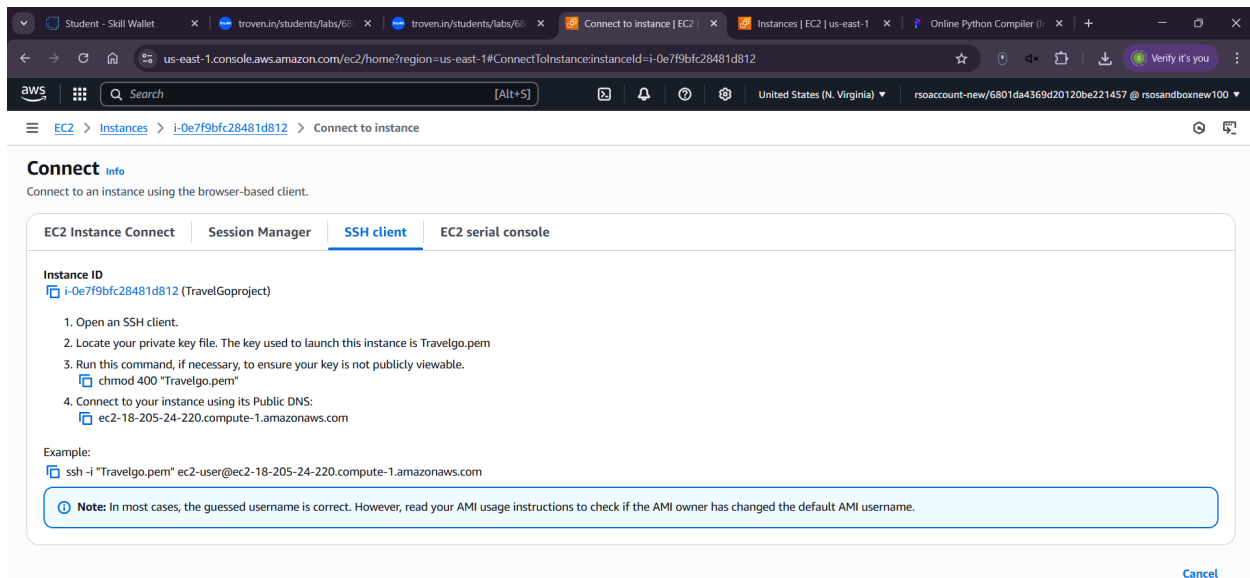
Modify IAM ROLE

Attach the IAM role Studentuser to your EC2 instance to grant secure access to AWS services like DynamoDB and SNS. This avoids hardcoding credentials and ensures your app functions with the required permissions. Make sure the role includes all necessary policies for smooth integration.



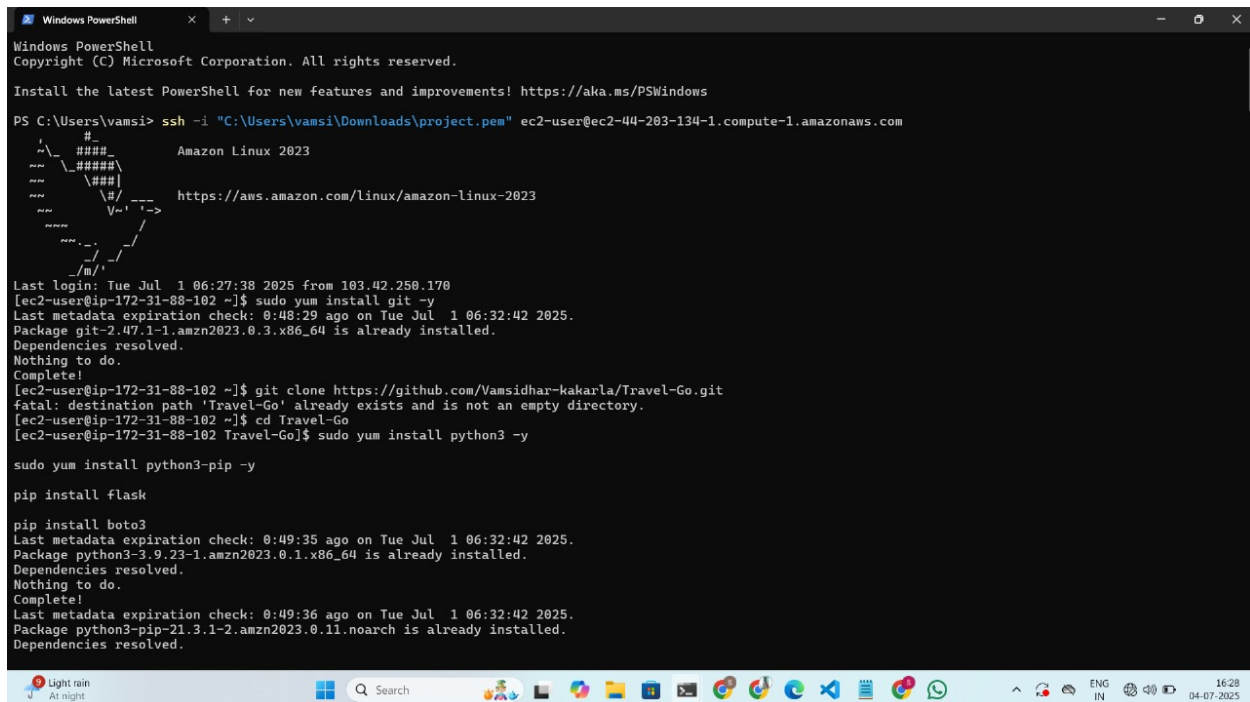


Wait until the confirmation message appears indicating that the Studentuser role has been successfully attached to the instance. This confirms that all required permissions are now active. Once attached, proceed to connect to your EC2 instance and run your GitHub-hosted Flask application code.



The following are the terminal outputs after executing the commands shown below the image. These outputs indicate that the setup steps have been successfully carried out.

The commands summary will be provided at last.



- sudo yum install git -y
- git clone your_repository_url
- cd your_project_directory
- sudo yum install python3 -y
- sudo yum install python3-pip -y

```
Windows PowerShell
Total download size: 1.9 M
Installed size: 11 M
Downloading Packages:
(1/2): libxcrypt-compat-4.4.33-7.amzn2023.x 2.5 MB/s | 92 kB 00:00
(2/2): python3-pip-21.3.1-2.amzn2023.0.11.n 31 MB/s | 1.8 MB 00:00
-----
Total                                     20 MB/s | 1.9 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                : 1/1
  Installing                : libxcrypt-compat-4.4.33-7.amzn2023.x86_64 1/2
  Installing                : python3-pip-21.3.1-2.amzn2023.0.11.noarch 2/2
  Running scriptlet         : python3-pip-21.3.1-2.amzn2023.0.11.noarch 2/2
  Verifying                 : libxcrypt-compat-4.4.33-7.amzn2023.x86_64 1/2
  Verifying                 : python3-pip-21.3.1-2.amzn2023.0.11.noarch 2/2

Installed:
  libxcrypt-compat-4.4.33-7.amzn2023.x86_64
  python3-pip-21.3.1-2.amzn2023.0.11.noarch

Complete!
[ec2-user@ip-172-31-94-133 Travel-Go]$ pip install flask
Defaulting to user installation because normal site-packages is not writeable
Collecting flask
  Downloading flask-3.1.1-py3-none-any.whl (103 kB)
    | 103 kB 15.1 MB/s
Collecting itsdangerous==2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting markupsafe==2.1.1
  Downloading MarkupSafe-3.0.2-cp39-cp39-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (20 kB)
Collecting click>=8.1.3
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
    | 98 kB 12.3 MB/s
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting importlib-metadata>=3.6.0
  Downloading importlib_metadata-8.7.0-py3-none-any.whl (27 kB)
Collecting Jinja2>=3.1.2
```

```
Windows PowerShell
Python 3.9.23
[ec2-user@ip-172-31-88-102 Travel-Go]$ python app.p
-bash: python: command not found
[ec2-user@ip-172-31-88-102 Travel-Go]$ python app.py
-bash: python: command not found
[ec2-user@ip-172-31-88-102 Travel-Go]$ Python app.py
-bash: Python: command not found
[ec2-user@ip-172-31-88-102 Travel-Go]$ python app.py
-bash: python: command not found
[ec2-user@ip-172-31-88-102 Travel-Go]$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.88.102:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 526-078-484
103.42.250.170 - - [01/Jul/2025 08:30:33] "GET / HTTP/1.1" 200 -
103.42.250.170 - - [01/Jul/2025 08:30:34] "GET /static/images/travel-bg.jpg HTTP/1.1" 404 -
103.42.250.170 - - [01/Jul/2025 08:30:34] "GET /favicon.ico HTTP/1.1" 404 -
103.42.250.170 - - [01/Jul/2025 08:30:39] "GET /login HTTP/1.1" 200 -
103.42.250.170 - - [01/Jul/2025 08:30:43] "GET /register HTTP/1.1" 200 -
103.42.250.170 - - [01/Jul/2025 08:30:58] "POST /register HTTP/1.1" 500 -
Traceback (most recent call last):
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
  File "/home/ec2-user/Travel-Go/app.py", line 50, in register
    existing = users_table.get_item(Key={'email': email})
```

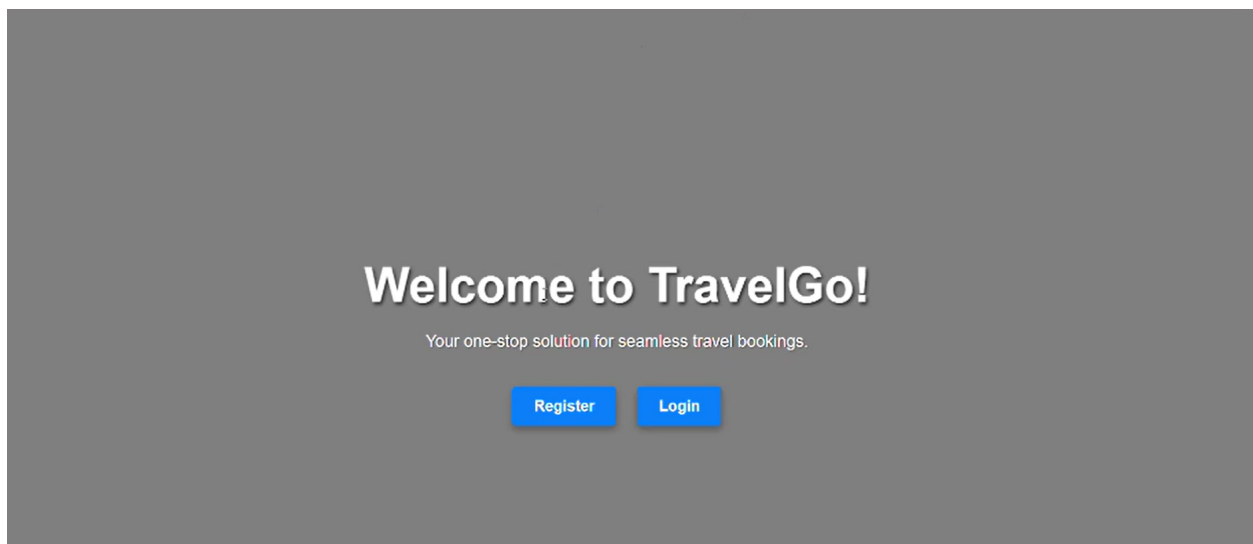
- pip install flask
- pip install boto3
- python3 app.py

Deployment Steps Summary:

- sudo yum install git -y
- git clone your_repository_url
- cd your_project_directory
- sudo yum install python3 -y
- sudo yum install python3-pip -y
- pip install flask
- pip install boto3
- python3 app.py

User Interface Screens:

- Homepage:



- Register and Login Portals:

The image displays two screenshots of the TravelGo web application's user authentication portals, set against a purple gradient background.

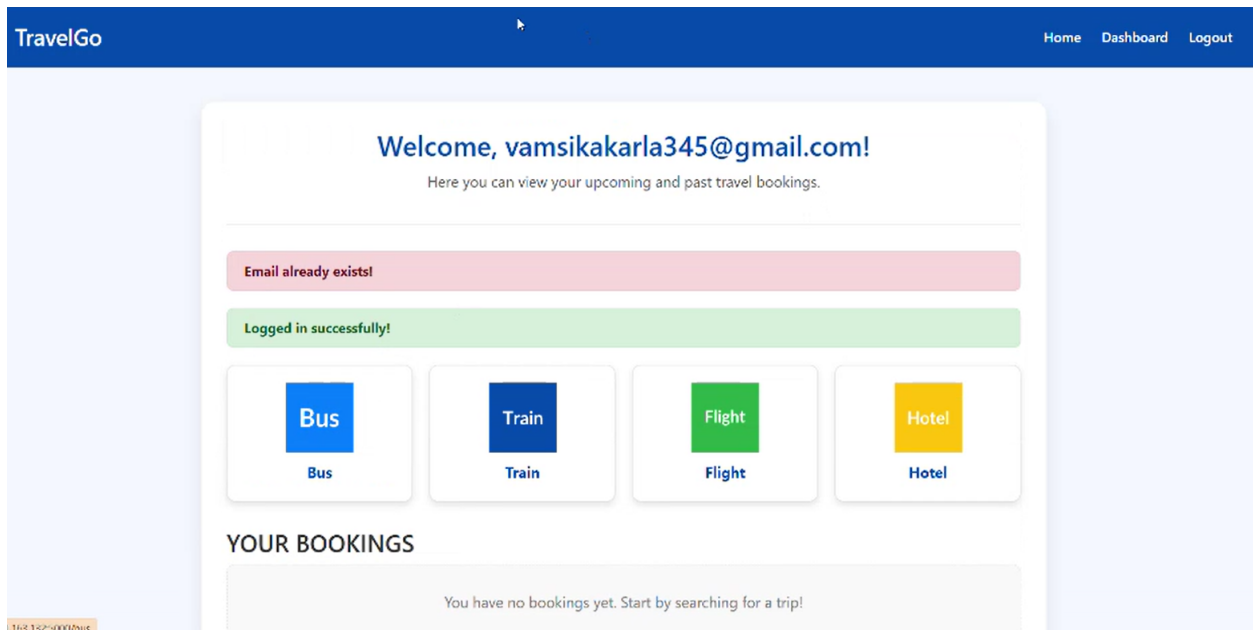
The top screenshot shows the "Register for TravelGo" form. It includes input fields for a name (Kakaria Vamsidhar), an email address (vamsikakaria345@gmail.com), and two password fields, each with a lock icon and three dots indicating masked text. A prominent purple "Register" button is at the bottom, with a link "Already have an account? Login here" below it.

The bottom screenshot shows the "Login to TravelGo" form. It features input fields for an email address (vamsikakaria345@gmail.com) and a password field (masked with three dots). A purple "Login" button is positioned below these fields, with a link "Don't have an account? Register Here" at the bottom.

- Dashboard View:

The dashboard serves as the central hub for users to access all travel services in one place. It displays a personalized greeting along with quick navigation cards for Bus, Train, Flight, and Hotel bookings. A success alert confirms the user's login, enhancing the user experience. Below, recent and upcoming bookings are listed with full travel details and cancellation options. This intuitive layout ensures users can manage their journeys effortlessly and efficiently.

Lets have a look at my dashboard page!!!

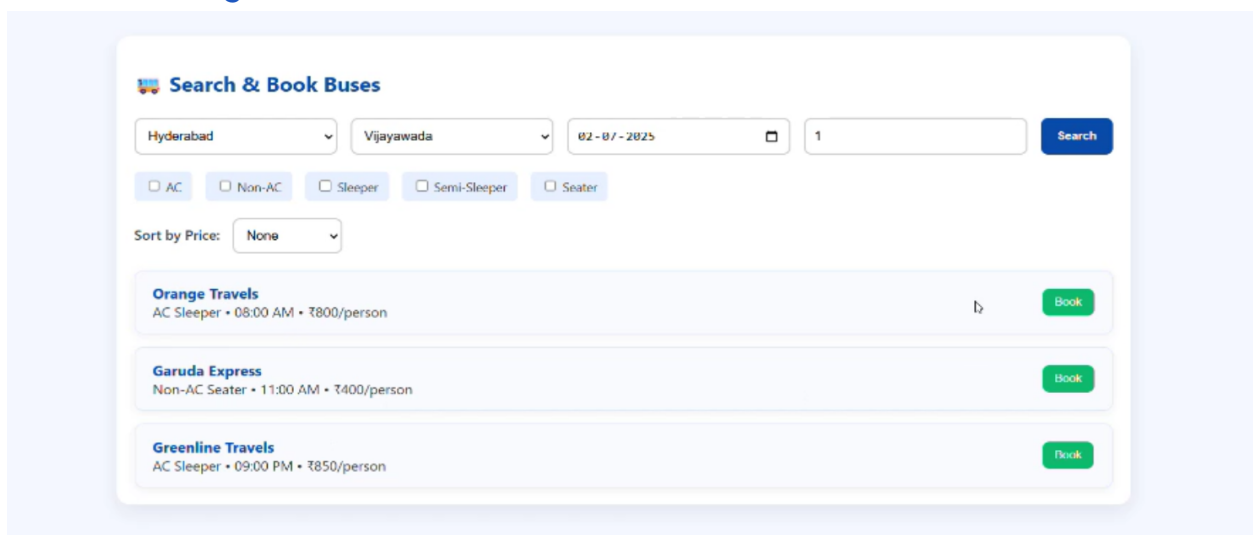


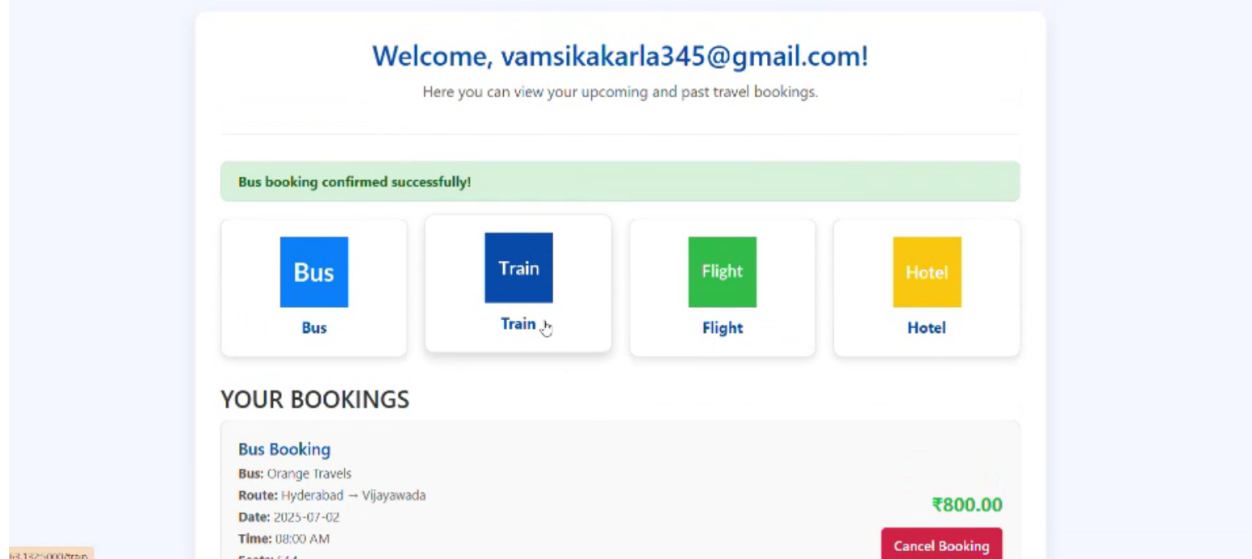
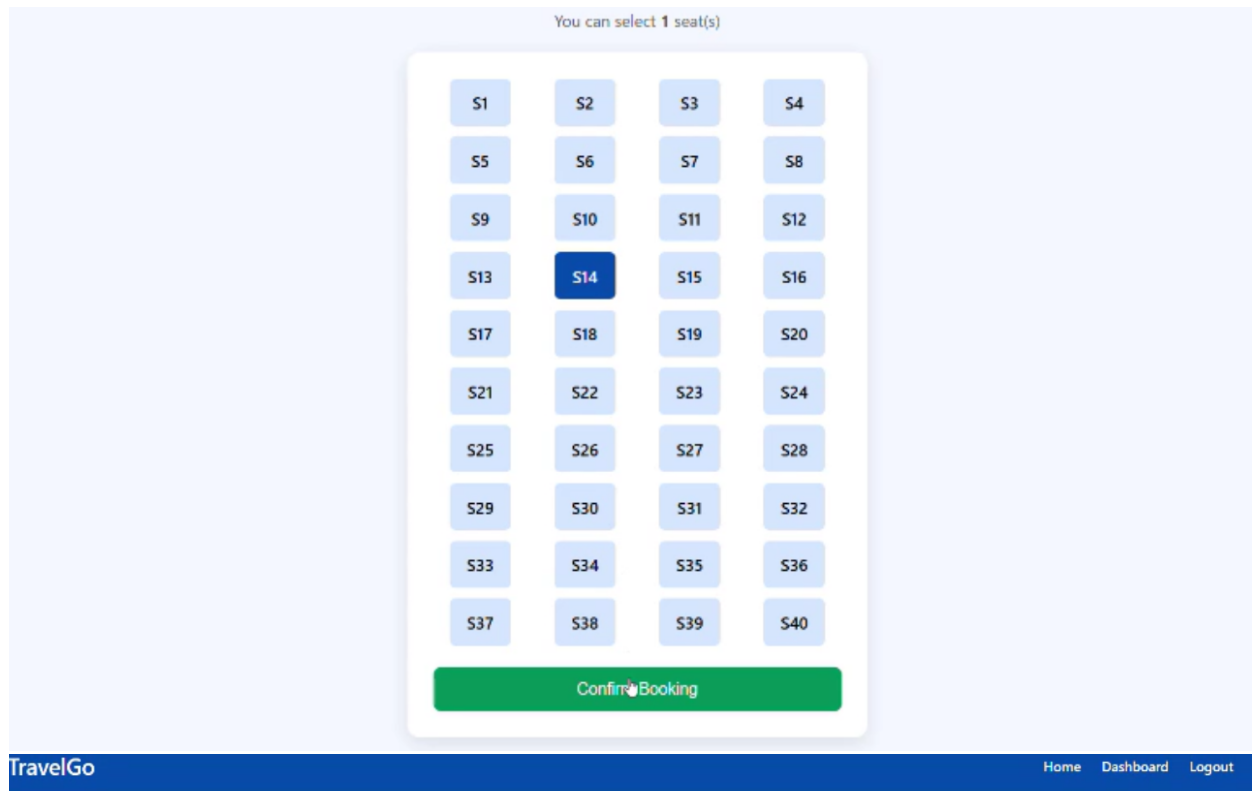
• Booking Tabs: Bus, Train, Flight, Hotel

☒ For demonstration purposes, a seat selection section has also been included in the bus booking module. This allows users to choose their preferred seats during the booking process, enhancing the overall user experience.

This interactive feature simulates real-world booking platforms and showcases the system's dynamic capabilities. It also helps users visualize the layout before confirming their reservation.

• Bus booking:





• Train booking:

✎ The train booking module enables users to search and reserve train tickets by selecting routes, dates, and times. It fetches real-time data and displays available options tailored to user input. Once a booking is made, the system confirms the reservation and stores the details in the database. Bookings are reflected instantly on the user dashboard for easy tracking. This module ensures a seamless and efficient booking experience for

train travelers.

Search & Book Trains

Hyderabad

Delhi

27-06-2025

1

Search

Duronto Express (12285)

From: Hyderabad **To:** Delhi

Departure: 07:00 AM **Arrival:** 05:00 AM

Date: 2025-06-27

Price per person: ₹1800

Total for 1: ₹1800

Book Now

TravelGo

HomeDashboardLogout

✓ Confirm Train Booking

Train booking confirmed successfully!

Booking Details:

Train Name: Duronto Express (12285)

Route: Hyderabad to Delhi

Date: 2025-06-27

Departure: 07:00 AM **Arrival:** 05:00 AM

Type:

Passengers: 1

Total Price: ₹1,800.00

Confirm Booking

Go Back to Search

• Flight booking:

➤The flight booking section enables users to search and reserve flights quickly based on their preferred source, destination, and date. Upon entering the details, the system

TravelGo

Search & Book Flights

Hyderabad ▾Mumbai ▾02. 07. 2025📅1

Search

IndiGo 6E-234
Hyderabad → Mumbai
2025-07-02 | 08:00 - 09:30

₹3500 x 1 = ₹3500
Book

Confirm Your Flight Booking

Airline: IndiGo (6E-234)
Route: Hyderabad → Mumbai
Date: 2025-07-02
Departure: 08:00
Arrival: 09:30
Passengers: 1
Price/person: ₹3500
Total Price: ₹3500

Confirm Booking

TravelGo

Search & Book Flights

Hyderabad ▾Mumbai ▾02. 07. 2025 📅1🔍 Search

IndiGo 6E-234
Hyderabad → Mumbai
2025-07-02 | 08:00 - 09:30

₹3500 x 1 = ₹3500Book

Confirm Your Flight Booking

Airline: IndiGo (6E-234)
Route: Hyderabad → Mumbai
Date: 2025-07-02
Departure: 08:00
Arrival: 09:30
Passengers: 1
Price/person: ₹3500
Total Price: ₹3500
Confirm Booking

- Hotel booking:

✶ The hotel booking section allows users to search and reserve accommodations based on their destination and travel dates. The interface displays available hotels with details such as location, price, and amenities. Users can easily compare options and proceed with booking in just a few clicks. Once confirmed, the booking details appear on the dashboard for easy tracking. The process is designed to be seamless, intuitive, and efficient for all types of travelers.



Book Your Perfect Stay

Hyderabad

02-07-2025

03-07-2025

1

1

Search

☐ 5-Star

☐ 4-Star

☐ 3-Star

☐ WiFi

☐ Pool

☐ Parking

Sort by:

None

Taj Falaknuma Palace

Hyderabad • 5-Star • ₹25000/night

Amenities: WiFi, Pool, Parking, Restaurant

Book

The Park

Hyderabad • 3-Star • ₹5500/night

Amenities: WiFi

Book

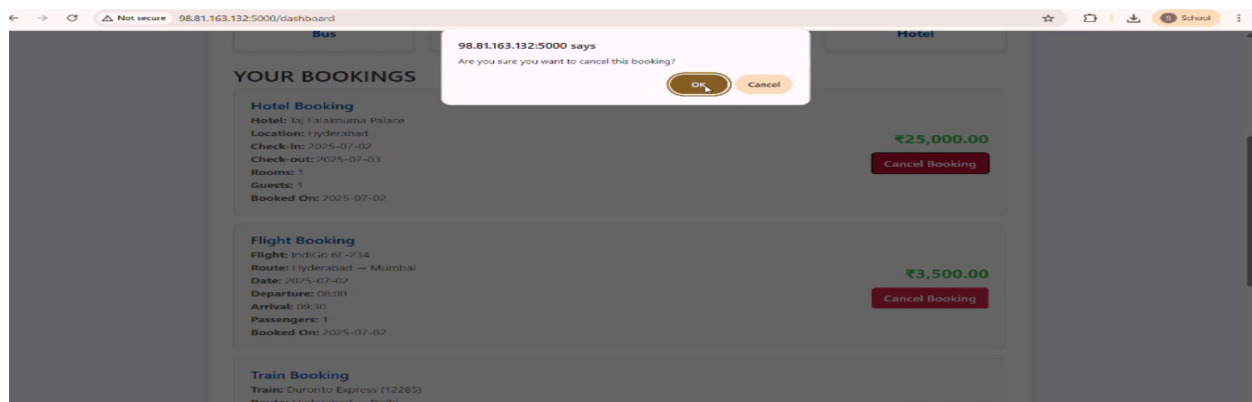
✧ At the bottom of the dashboard, users can view a summary of all their bookings, including travel details, dates, and status. A dedicated “Cancel” button is provided next to each entry, allowing users to easily cancel any upcoming reservations if needed.

This section provides a centralized view for managing bookings efficiently. It ensures transparency by displaying every confirmed ticket in an organized format. The cancel feature updates the backend in real time, removing the booking from the list and database. This functionality enhances user control and flexibility while planning or modifying travel. It also improves overall convenience by reducing the need to revisit individual booking sections.

All Bookings:

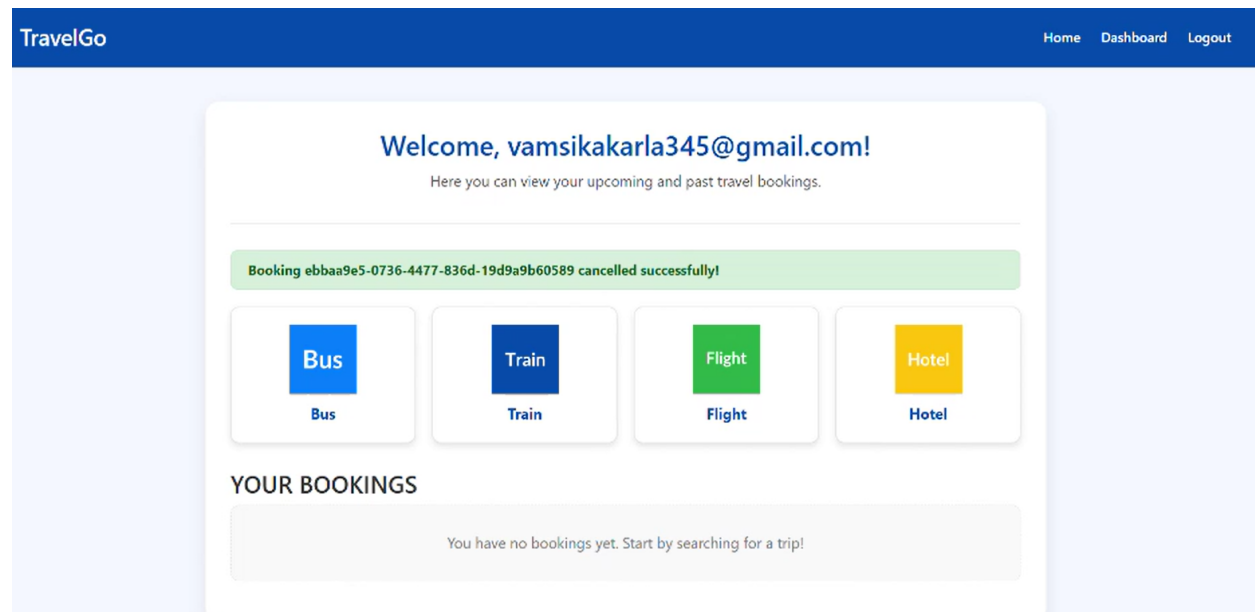


Cancel Bookings:



Once a booking is cancelled, a confirmation message appears indicating the cancellation was successful. This real-time feedback reassures the user that their action has been completed without issues. It also helps avoid confusion or repeated attempts. The booking entry is immediately removed from the dashboard view. This

seamless flow enhances the overall usability and responsiveness of the application. Let's have a look at the page indicating the cancellation was successful.



Final Conclusion – Elevating Travel with TravelGo:

TravelGo stands as a dynamic and cloud-powered solution that redefines how users plan and book their journeys. By seamlessly integrating Flask for backend logic, AWS EC2 for scalable deployment, DynamoDB for reliable data storage, and SNS for real-time notifications, the platform delivers a smooth, responsive, and user-centric experience.

From login to booking, and from instant alerts to effortless cancellations, every feature is built with efficiency and accessibility in mind. Whether it's buses, trains, flights, or hotels—TravelGo offers a unified platform that adapts to diverse travel needs. Its modern architecture ensures high performance, security, and scalability even under peak load conditions.

 **With TravelGo, travel planning is no longer a hassle—it's an experience.**
Smart. Fast. Reliable. That's the future of travel.